

Underfitting

- Technical perspective: Underfitting means the model has high bias but low variance.
- Outcome: Weak results on both training and validation data.
- Example: Using a shallow neural network for a complex image classification task.

How to recognize it:

- Both training and validation accuracy remain low
- The loss stays high after many epochs

How to Fix Underfitting

To reduce Underfitting:

- Increase model complexity (more layers, deeper networks).
- Train for more epochs.
- Decrease the strength of regularization.

Overfitting

- Technical perspective: Overfitting means the model has low bias but high variance.
- Outcome: Excellent results on training data, disappointing results on validation data.
- Example: A very deep network with many parameters on a small dataset, leading to memorization instead of learning general patterns.

How to recognize it:

- Training accuracy is extremely high.
- Validation accuracy is much lower.
- The gap between the two curves grows with training time.

Why Validation Accuracy Lags Behind Training Accuracy

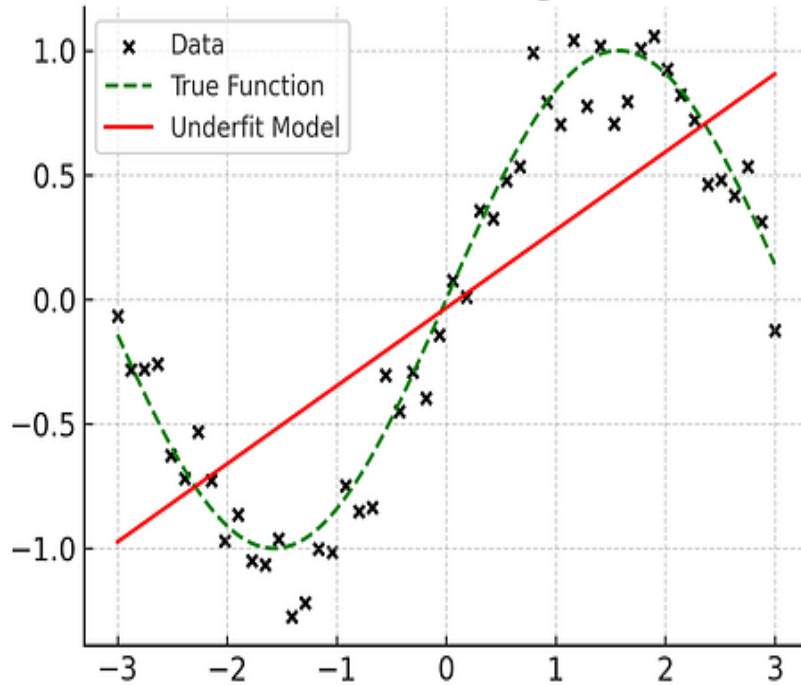
Training accuracy: measured on the data the model has already seen and optimized for.

Validation accuracy: measured on unseen data, where the model must generalize.

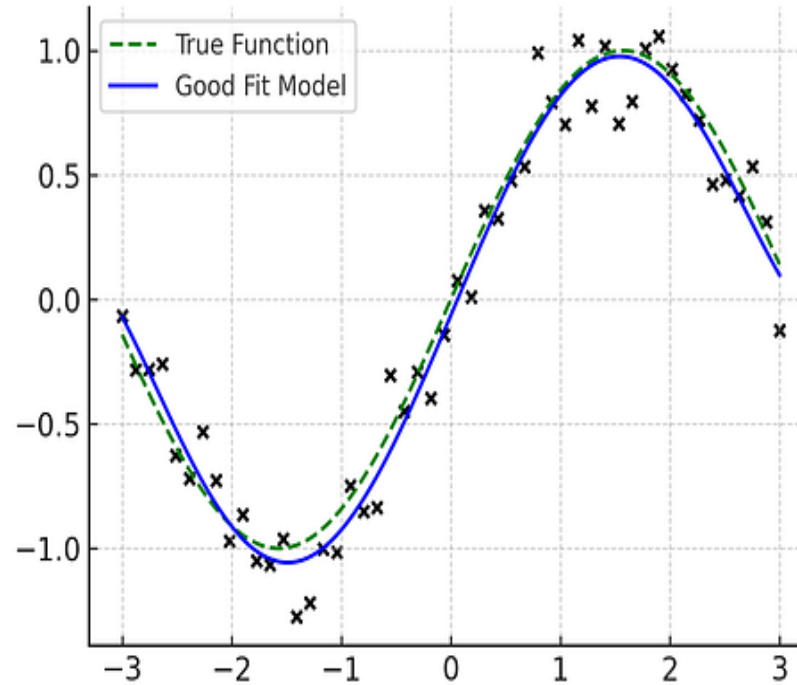
So naturally, validation accuracy almost always will be lower than training accuracy.

Visual Intuition

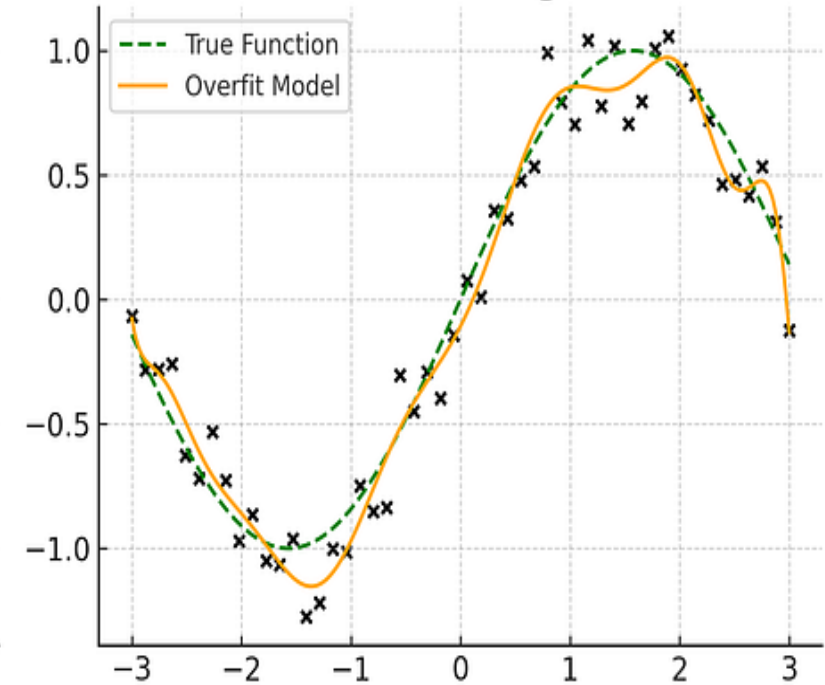
Underfitting



Good Fit



Overfitting



How to Fix Overfitting

To reduce Overfitting:

- Add regularization (L1, L2, Dropout).
- Use data augmentation to make training examples more varied.
- Apply early stopping during training
- Simplify the model if it's too deep or wide.
- Collect more diverse data

Note: Both bias and variance contribute to the mean squared error

What is Regularization

- Regularization is a technique used in deep learning models to prevent overfitting and improve their reliability.
- Regularization helps to:
 - Reduce overfitting
 - Improves generalization
 - Prioritize the most important features in the data.

How Regularization avoids overfitting?

- Regularization avoids overfitting by adding a penalty to the loss function that the model tries to minimize:

$$\textit{Regularization} = \textit{Loss Function} + \textit{Penalty}$$

- The penalty term discourages the model from assigning too much importance to any single parameter or feature, effectively reducing the complexity of the model.

Common Regularization Techniques

1. L1 Regularization (Lasso Regularization)
2. L2 Regularization (Ridge Regularization)
3. Elastic Net
4. Dropout
5. Data Augmentation

1. L1 Regularization (Lasso Regularization)

- For l_1 regularization we have,

$$\widetilde{\mathcal{L}}(w) = \mathcal{L}(w) + \alpha \|w\|$$

- For SGD (or its variants), we are interested in

$$\nabla \widetilde{\mathcal{L}}(w) = \nabla \mathcal{L}(w) + \alpha \text{sign}(w)$$

$$\text{sign}(w_j) = \begin{cases} +1 & w_j > 0 \\ -1 & w_j < 0 \\ 0 & w_j = 0 \end{cases}$$

- Update rule:

$$w_{t+1} = w_t - \eta \nabla \mathcal{L}(w_t) - \eta \alpha \text{sign}(w_t)$$

Where:

- $\mathcal{L}(w)$: empirical loss
- w : weight parameters of the model
- $\|w\|$: L1 norm of weights, $\|w\| = |w_1| + |w_2| + \dots + |w_m| = \sum_{j=1}^m |w_j|$
- α : regularization parameter

2. L2 Regularization (Ridge Regularization)

- For l_2 regularization we have,

$$\widetilde{\mathcal{L}}(w) = \mathcal{L}(w) + \frac{\alpha}{2} \|w\|^2$$

- For SGD (or its variants), we are interested in

$$\nabla \widetilde{\mathcal{L}}(w) = \nabla \mathcal{L}(w) + \alpha w$$

- Update rule:

$$w_{t+1} = w_t - \eta \nabla \mathcal{L}(w_t) - \eta \alpha w_t$$

Where:

- $\mathcal{L}(w)$: empirical loss
- w : weight parameters of the model
- $\|w\|^2$: squared L2 norm of weights,
 $\|w\|^2 = w_1^2 + w_2^2 + \dots + w_m^2 = \sum_{j=1}^m w_j^2$
- α : regularization parameter

2. L2 Regularization (Ridge Regularization)

- Unlike LASSO, Ridge does not drive coefficients to zero, but rather shrinks them toward zero while keeping all features in the model
- This helps prevent overfitting while maintaining all features for interpretation.
- Additionally, Ridge requires careful tuning of the regularization parameter (Disadvantage)
- The method also assumes that all features are equally important for regularization, which may not be appropriate for datasets where some features are known to be more relevant than others (Disadvantage)

3. Elastic Net

- Elastic Net is a regularization technique that combines:
 - L1 regularization (Lasso)
 - L2 regularization (Ridge)
- This technique is especially useful for:
 - High-dimensional data
 - Correlated predictors
 - Feature selection and regularization
- Select features like Lasso does, but without arbitrarily tossing away correlated ones.
- Handle multicollinearity like Ridge, ensuring that no important information is lost.

3. Elastic Net

- For Elastic net regularization we have,

$$\tilde{\mathcal{L}}(w) = \mathcal{L}(w) + \lambda_1 \|w\|_1 + \frac{\lambda_2}{2} \|w\|^2$$

- For SGD (or its variants), we are interested in

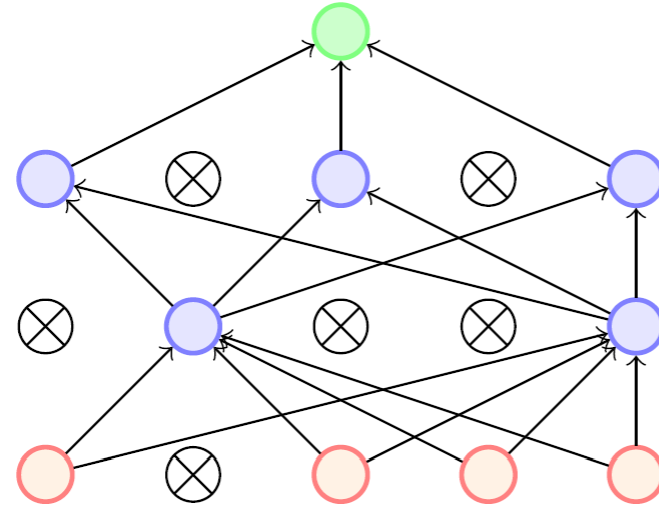
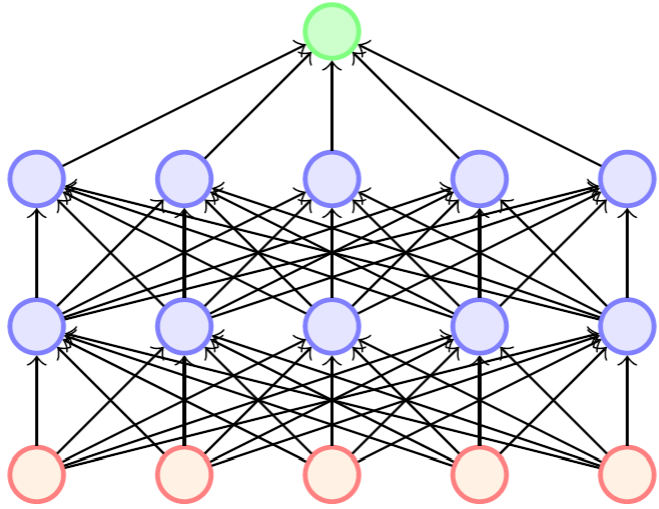
$$\nabla \tilde{\mathcal{L}}(w) = \nabla \mathcal{L}(w) + \lambda_1 \text{sign}(w) + \lambda_2 w$$

$$\text{sign}(w_j) = \begin{cases} +1 & w_j > 0 \\ -1 & w_j < 0 \\ 0 & w_j = 0 \end{cases}$$

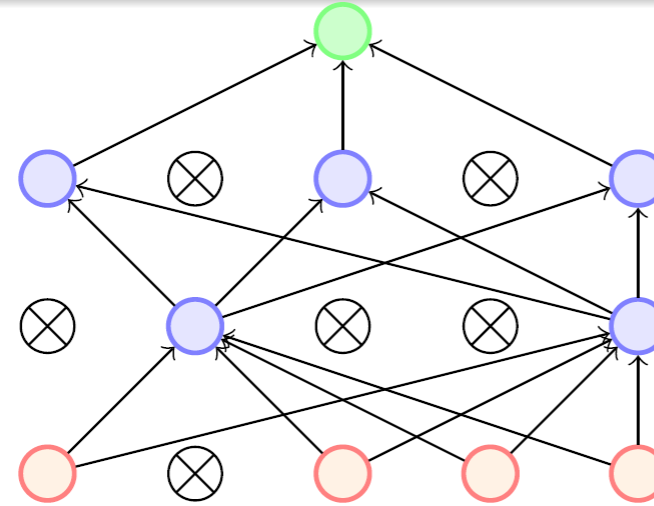
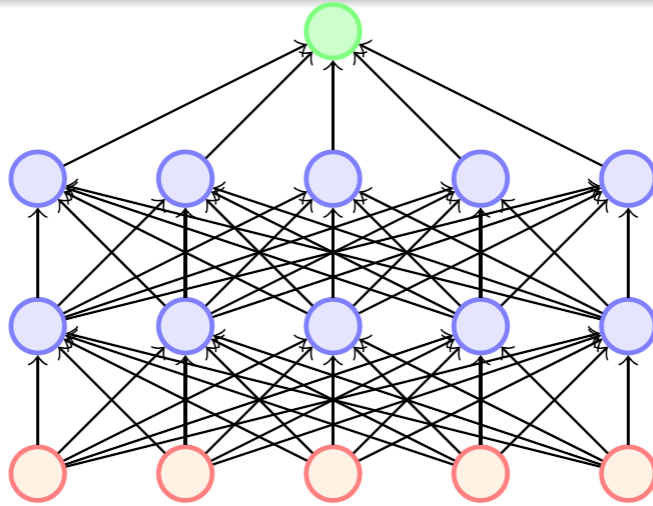
- Update rule:

$$w_{t+1} = w_t - \eta \nabla \mathcal{L}(w_t) - \eta \lambda_1 \text{sign}(w_t) - \eta \lambda_2 w_t$$

4. Dropout



- Dropout refers to dropping out units
- Temporarily remove a node and all its incoming/outgoing connections resulting in a thinned network
- Each node is retained with a fixed probability (typically $p = 0.5$) for hidden nodes and $p = 0.8$ for visible nodes



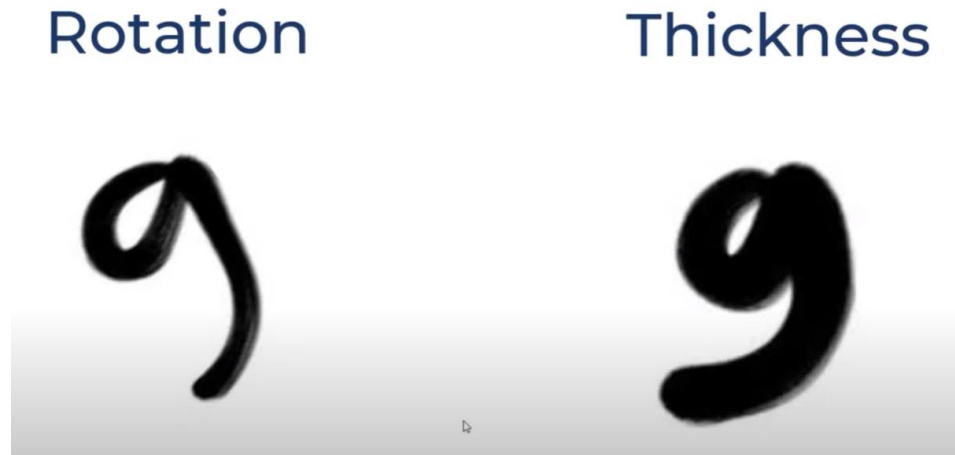
- Suppose a neural network has n nodes
- Using the dropout idea, each node can be retained or dropped
- For example, in the above case we drop 5 nodes to get a thinned network
- Given a total of n nodes, what are the total number of thinned networks that can be formed? 2^n

Note: Dropout is applied

- Only during training
- During testing/inference, all neurons are used, but outputs are scaled properly

5. Data Augmentation

- CNN can not handle rotation and scale by itself

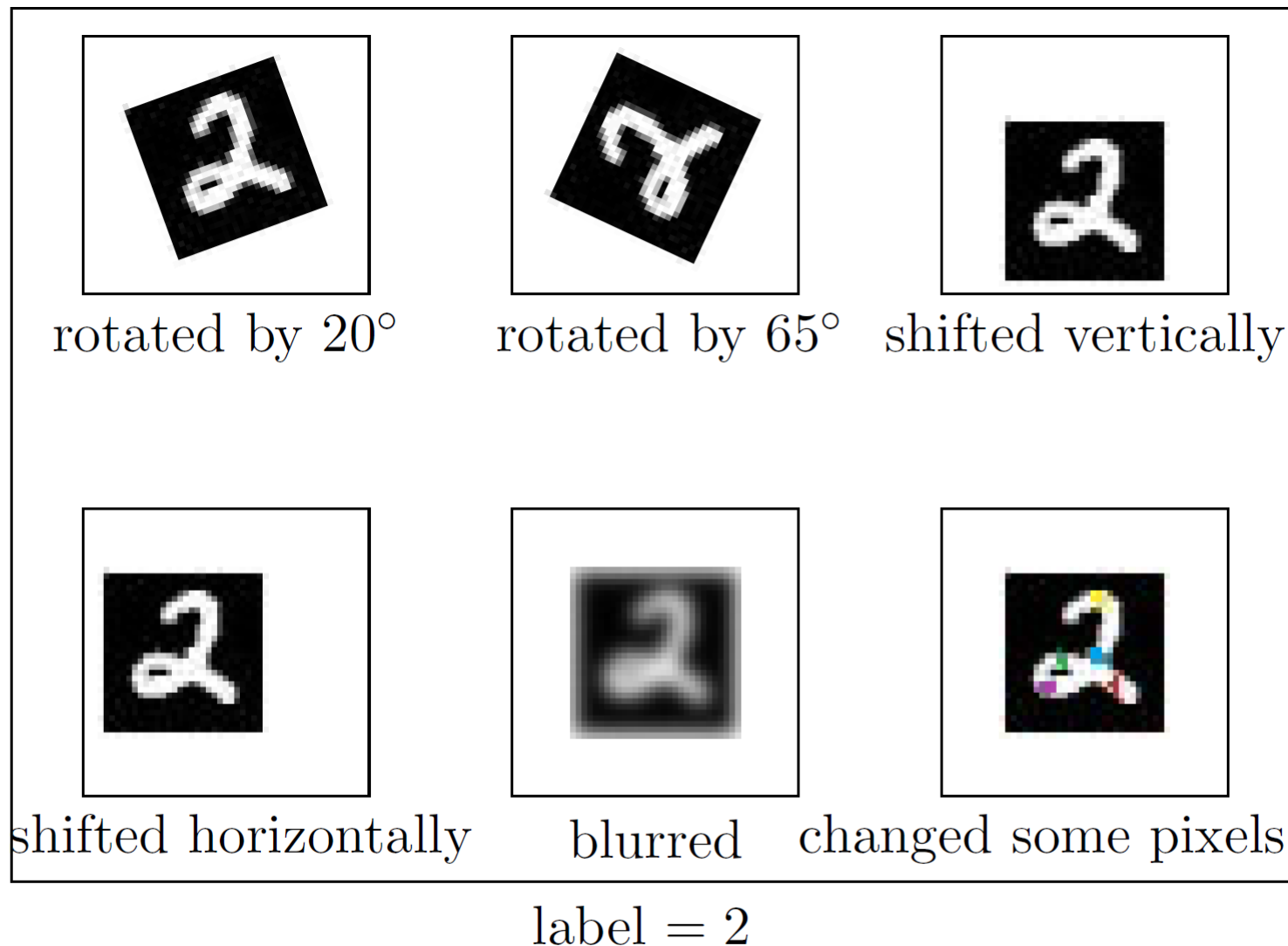


- Due to this, training dataset should have rotated and scaled sample
- If it does not have than pick some of the sample from training data set and rotate and scale them using Data Augmentation or Spatial Transformer Networks (STN)



label = 2

[given training data]
We exploit the fact that certain transformations to the image do not change the label of the image.



[augmented data = created using some knowledge of the task]

Why Weight Initialization Matters?

- Weight initialization is the process of setting the starting values for a neural network's weights before training.
- These weights help the network learn by adjusting them through each iteration. Poor initialization can lead to issues like:
 - **Vanishing Gradient Problem:** Gradients become very small, slowing down or stopping training.
 - **Exploding Gradient Problem:** Gradients become excessively large, leading to instability.
 - **Slow Convergence:** Training takes longer to reach an optimal solution.
- Proper weight initialization can reduce these problems, making the network converge faster and perform better.

Types of Weight Initialization Techniques?

1. **Zero Initialization:** All the weights are assigned zero

- All neurons learn the same feature during each iteration
- The model won't learn anything meaningful
- Same issue will occur in constant Initialization

2. **Random Initialization:** Assigns weights randomly to avoid the zero-initialization Problem

- Problems such as Overfitting, Vanishing Gradient Problem, Exploding Gradient Problem might occur.

Types of Weight Initialization Techniques?

3. Xavier/Glorot Initialization:

- Xavier initialization is a technique used to initialize the weights of neural network which solves the problem of vanishing and exploding gradients
- Xavier initialization is particularly useful when using the sigmoid or tanh activation functions.

3.1 Xavier uniform initializer: the weights are assigned from values of a uniform distribution

$$w_i \sim U \left[-\sqrt{\frac{6}{fan_in + fan_out}}, \sqrt{\frac{6}{fan_in + fan_out}} \right]$$

Where, fan_in is the number of input units and fan_out is the number of output units)

Types of Weight Initialization Techniques?

3.2 Xavier normal initializer: the weights are assigned from values of a normal distribution centered on 0.

$$w_i \sim \mathcal{N} \left(0, \frac{2}{fan_in + fan_out} \right)$$

So the standard deviation is:

- Mean = 0
- Variance = $\frac{2}{fan_in + fan_out}$

$$\sigma = \sqrt{\frac{2}{fan_in + fan_out}}$$

In a normal distribution, 99.7% of all values fall within 3 standard deviations of the mean. So the range of weights will be $[-3\sigma, +3\sigma]$

Types of Weight Initialization Techniques?

4.1 He Uniform Initialization: the weights are assigned from values of a uniform distribution

$$w_i \sim U \left[-\sqrt{\frac{6}{fan_in}}, \sqrt{\frac{6}{fan_out}} \right]$$

- He Uniform initialization is suitable for layers where ReLU activation function is used

Types of Weight Initialization Techniques?

4.2 He Normal Initialization: the weights are assigned from values of a normal distribution

$$w_i \sim \mathcal{N}\left(0, \frac{2}{fan_in}\right)$$

- He Normal initialization is suitable for layers where ReLU activation function is used

References

- NPTEL/YouTube lecture series on Deep Learning by Mitesh M. Khapra (IIT Madras)
- Material compiled and adapted from various educational websites, including online tutorials, blogs, and open learning platforms.